

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Developing Web Applications-Spring Web MVC Framework

Table of Contents

Learning Outcome:	3
1. Introduction to Spring Web MVC Framework	4
Key Features and Benefits of Using Spring MVC	4
2. Spring MVC Auto-Configuration	6
Leveraging Spring Boot Starters for Spring MVC	6
3. <code>HttpMessageConverters</code>	6
4. <code>MessageCodesResolver</code>	7
Configuring a <code>MessageCodesResolver</code>	8
Resolving Error Messages and Validation Codes	9
5. Static Content Handling	10
Configuring Resource Mapping and Handling	10
Caching and Performance Considerations	11
6. Path Matching and Content Navigation	12
URL Patterns and Request Mapping in Spring MVC	12
Content Navigation Techniques for User-Friendly URLs	13
7. <code>ConfigurableWebBindingInitializer</code>	13
Data Binding and Type Conversion in Spring MVC	14
Handling Complex Data Binding Scenarios	14
8. Spring HATEOAS	15
Building Hypermedia-Driven RESTful APIs using Spring HATEOAS	16
Linking and Navigating Resources	17
9. CORS Support	17
Handling Cross-Origin Requests and Security Considerations	19

UNIT 11: Developing Web Applications - Spring Web MVC Framework

Objectives:

In this Unit you will learn to –

- Understand the fundamental concepts and principles of the Spring Web MVC Framework.
- Learn how to leverage Spring MVC Auto-Configuration for quick and effortless setup of a Spring MVC application.
- Understand the importance of MessageCodesResolver in resolving error messages and handling validation.
- Content navigation techniques for building user-friendly URLs and routing requests.
- Understand the concepts and usage of Spring HATEOAS for building hypermedia-driven RESTful APIs.
- Learn about Cross-Origin Resource Sharing (CORS) and how to configure CORS support in a Spring MVC application.

Learning Outcome:

Use Spring Web for developing a web application.

11. Developing Web Applications - Spring Web MVC Framework

1. Introduction to Spring Web MVC Framework

The Spring Web MVC Framework is a powerful and flexible framework for building web applications based on the Model-View-Controller (MVC) architectural pattern. It provides a structured approach to designing and developing web applications by separating concerns and promoting modularity, testability, and maintainability.

The key components of the Spring MVC architecture are:

- **Model:** The model represents the data and business logic of the application. It encapsulates the application's state and provides methods for manipulating and accessing the data.
- **View:** The view is responsible for rendering the user interface. It generates the HTML, XML, JSON, or other types of responses that are sent back to the client. The view receives data from the model and presents it to the user.
- **Controller:** The controller handles user requests and acts as an intermediary between the model and the view. It processes incoming requests, interacts with the model to retrieve or update data, and selects the appropriate view to render the response.

Key Features and Benefits of Using Spring MVC

- **Flexibility:** Spring MVC offers great flexibility in terms of configuration and customization. It allows developers to choose between XML-based configuration or annotation-based configuration, based on their preferences. Additionally, it provides a wide range of extension points and hooks for adding custom functionality.
- **Separation of Concerns:** Spring MVC promotes the separation of concerns by dividing an application into distinct layers. The model represents the data and business logic, the view handles the presentation and user interface, and the controller manages the flow and coordination between them. This separation

improves code modularity, testability, and maintainability.

- **Request-Driven:** Spring MVC follows a request-driven approach, where each incoming request is mapped to a specific controller and method based on the defined URL patterns. This allows for flexible request handling and routing, enabling developers to create clean and user-friendly URLs.
- **Extensive Validation Support:** Spring MVC provides robust validation support through its integration with Java Bean Validation API (JSR 380). It simplifies input validation and error handling by automatically validating user input and binding validation errors to the model.
- **Integration with other Spring Projects:** Spring MVC seamlessly integrates with other Spring projects, such as Spring Core, Spring Data, Spring Security, and more. This enables developers to leverage the full power of the Spring ecosystem and build comprehensive enterprise applications.
- **Testability:** Spring MVC's architecture promotes testability by allowing easy unit testing of controllers and models. The framework provides support for writing unit tests using mock objects and dependency injection, making it easier to test individual components in isolation.
- **Internationalization and Localization:** Spring MVC offers built-in support for internationalization and localization. It provides mechanisms for handling multiple languages, managing resource bundles, and resolving messages based on the user's locale.
- **Integration with Front-End Technologies:** Spring MVC integrates smoothly with front-end technologies such as HTML, CSS, JavaScript, and popular JavaScript frameworks like Angular, React, or Vue.js. It allows developers to build rich and interactive web applications by combining the power of Spring MVC on the server-side with modern front-end frameworks on the client-side.

Spring MVC is a mature and widely adopted framework that provides developers with the tools and capabilities to build scalable, maintainable, and robust web applications. Its modular architecture, flexibility, and integration capabilities make it a popular choice for developing enterprise-grade web applications.

2. Spring MVC Auto-Configuration

Auto-configuration is a key feature of Spring Boot that aims to simplify the setup and configuration of Spring applications. It leverages sensible default settings and automatic configuration based on the dependencies present in the classpath. Spring Boot provides auto-configuration for various modules, including Spring MVC.

Auto-configuration in Spring Boot eliminates the need for manual configuration by automatically configuring beans and components based on the application's classpath. When an application starts up, Spring Boot scans the classpath for specific libraries, annotations, or properties and configures the necessary infrastructure automatically.

Leveraging Spring Boot Starters for Spring MVC

Spring Boot starters are a set of dependencies that bring in all the necessary libraries and configurations required for a specific technology or feature. To enable Spring MVC auto-configuration, you need to include the `spring-boot-starter-web` starter dependency in your project.

By including this starter dependency in your application, Spring Boot automatically configures the essential components required for Spring MVC, such as `DispatcherServlet`, `ViewResolver`, and other necessary infrastructure. It also configures sensible default settings and conventions for Spring MVC, making it easier to get started with building web applications.

3. HttpMessageConverters

`HttpMessageConverters` play a crucial role in Spring MVC for handling request and response data in different formats. They are responsible for converting the data between Java objects and the representation formats supported by HTTP, such as JSON, XML, and others. `HttpMessageConverters` enable seamless data binding and serialization/deserialization between the application and clients.

When a client sends a request to a Spring MVC application, the `HttpMessageConverters` come into play during request processing. They handle the conversion of the request payload into Java objects that can be processed by the application's controllers and services. Similarly, during response processing, the `HttpMessageConverters` convert the Java objects returned by the application into the desired representation format that the client expects.

Spring MVC provides out-of-the-box support for various message formats through different implementations of `HttpMessageConverters`. Some commonly supported message formats include:

- **JSON (JavaScript Object Notation):** JSON is a lightweight data interchange format widely used for representing structured data. Spring MVC supports JSON through converters such as `MappingJackson2HttpMessageConverter`, `GsonHttpMessageConverter`, and `Jackson2CborHttpMessageConverter`.
- **XML (eXtensible Markup Language):** XML is a markup language used for representing structured data. Spring MVC supports XML through converters like `MappingJackson2XmlHttpMessageConverter`, `Jaxb2RootElementHttpMessageConverter`, and `MarshallingHttpMessageConverter`.
- **Form Data:** Spring MVC provides converters to handle form data submitted through HTML forms, such as `FormHttpMessageConverter` and `StringHttpMessageConverter`.
- **Text:** `HttpMessageConverters` also support plain text formats, including `StringHttpMessageConverter`, `ByteArrayHttpMessageConverter`, and `ResourceHttpMessageConverter`.

4. MessageCodesResolver

`MessageCodesResolver` is a component in Spring MVC that is responsible for resolving error messages and validation codes during error handling and validation processes. It plays a crucial role in providing meaningful error messages to users and developers, facilitating better error handling and troubleshooting.

In Spring MVC, message codes are unique identifiers associated with error messages. They serve as a standardized way to identify and categorize different types of errors that may occur during request processing or validation. Message codes help in decoupling error messages from the actual error handling code, making it easier to manage and customize error messages across the application.

The importance of message codes in error handling includes:

- **Localization:** Message codes allow for easy localization of error messages. By associating message codes with resource bundles or message source files, developers can provide error messages in multiple languages or customize them for different locales.
- **Centralized Error Handling:** Message codes enable centralized error handling by categorizing errors based on their nature or source. Developers can define specific error handling logic for different message codes, providing consistent and tailored error responses to clients.
- **Maintainability:** Message codes help separate the concerns of error handling from the business logic. They provide a clear distinction between error messages and code logic, making it easier to update or modify error messages without changing the underlying code.
- **Consistency:** Message codes promote consistency in error messages across the application. By using standardized message codes, developers can ensure that error messages are uniform and adhere to predefined conventions.

Configuring a MessageCodesResolver

In Spring MVC, you can configure a MessageCodesResolver to define the strategy for resolving error messages and validation codes. The configuration can be done at the application level or on a per-field basis.

To configure a MessageCodesResolver:

- **Application-level Configuration:** You can configure a MessageCodesResolver at the application level by implementing the MessageCodesResolver interface

and defining your custom logic for resolving message codes. Then, register the custom `MessageCodesResolver` implementation as a bean in the Spring application context.

- **Field-level Configuration:** Spring MVC provides a default implementation of the `MessageCodesResolver` interface called `DefaultMessageCodesResolver`. This resolver uses conventions to generate message codes based on field names, nested field paths, and validation errors. You can customize the default resolver or provide your own `MessageCodesResolver` implementation for field-level configuration.

Resolving Error Messages and Validation Codes

During error handling and validation processes in Spring MVC, the `MessageCodesResolver` is used to resolve error messages and validation codes.

The process typically involves the following steps:

- **Binding Errors:** When binding errors occur, such as data binding failures or type conversion errors, the `MessageCodesResolver` is invoked to resolve error messages for these errors. The resolver generates message codes based on the error type, field names, and nested field paths.
- **Validation Errors:** During validation, if validation errors occur, the `MessageCodesResolver` is used to resolve validation codes. The resolver generates message codes based on the validator, field names, and nested field paths.
- **Message Resolving:** The resolved message codes are then passed to the `MessageSource` (configured in the application) for actual message resolution. The `MessageSource` retrieves the appropriate error messages based on the message codes and the configured locale.
- **Displaying Error Messages:** The resolved error messages are typically displayed to the user as part of the error response or validation feedback.

5. Static Content Handling

In a Spring MVC application, static resources such as HTML files, CSS stylesheets, JavaScript files, and images are considered static content. Spring MVC provides built-in mechanisms to handle and serve static resources efficiently. This allows for seamless integration of static content with dynamic views and logic in the application.

To serve static resources in a Spring MVC application, you need to place the static content in designated directories. By default, Spring MVC looks for static resources in the following directories under the classpath:

- `/static`: Static resources placed in this directory will be served as is, without any modifications.
- `/public`: Resources in this directory are also served as is, similar to the `/static` directory.
- `/resources`: Resources in this directory are served as is, similar to the `/static` and `/public` directories.
- `/META-INF/resources`: Resources in this directory are typically found in JAR files and can be served as static content.

For example, if you have an HTML file named `index.html`, you can place it in the `/static` directory, and it will be accessible at the root URL (`/index.html`) of your application.

Configuring Resource Mapping and Handling

In addition to the default directories, you can also configure custom resource locations and define resource handlers to handle the mapping of URLs to static resources.

To configure resource mapping and handling in Spring MVC, you can use the `addResourceHandlers()` method of the `WebMvcConfigurer` interface.

Here's an example:

```

@Configuration
@EnableWebMvc

public class MvcConfig implements WebMvcConfigurer {

    @Override

    public void addResourceHandlers(ResourceHandlerRegistry registry) {

        registry.addResourceHandler("/my-resources/**")

            .addResourceLocations("classpath:/my-static-resources/");

    }

}

```

In the above example, any URL starting with /my-resources/ will be mapped to static resources located in the classpath:/my-static-resources/ directory.

Caching and Performance Considerations

Caching static resources can significantly improve the performance of a web application. Spring MVC provides caching support for static resources out of the box. By default, resources are cached based on their last-modified timestamps and ETag headers.

To enable caching and set cache-related properties, you can use the ResourceHandlerRegistration object returned by the addResourceHandlers() method.

For example:

```

@Override

public void addResourceHandlers(ResourceHandlerRegistry registry) {

    registry.addResourceHandler("/my-resources/**")

        .addResourceLocations("classpath:/my-static-resources/")

        .setCacheControl(CacheControl.maxAge(1,
            TimeUnit.HOURS).cachePublic());

}

```

In the above example, the resources under `/my-resources/` will be cached for 1 hour, and the Cache-Control header will be set to allow public caching.

6. Path Matching and Content Navigation

In a Spring MVC application, path matching and content navigation are crucial for handling and routing incoming requests. They play a significant role in defining URL patterns, mapping requests to appropriate controllers or handlers, and creating user-friendly and meaningful URLs.

URL Patterns and Request Mapping in Spring MVC

URL patterns are used to define the structure and format of URLs in a Spring MVC application. Spring MVC provides several mechanisms for mapping URLs to controllers or handlers:

- **Request Mapping Annotations:** Spring MVC offers various annotations, such as `@RequestMapping`, `@GetMapping`, `@PostMapping`, etc., that can be used to map specific URLs or URL patterns to methods in controllers. These annotations allow you to specify URL patterns, HTTP methods, request parameters, and more.
- **Ant-style Path Patterns:** Spring MVC supports Ant-style path patterns for flexible URL mapping. These patterns use wildcard characters (`*` and `**`) to match multiple path segments or any number of characters. For example, `/users/*` matches URLs like `/users/123`, `/users/abc`, etc.
- **Path Variables:** Path variables are used to capture dynamic parts of a URL and pass them as arguments to controller methods. They are denoted by curly braces (`{}`) in the URL pattern. For example, `/users/{id}` captures the `id` portion of the URL and makes it available as a method parameter.

Content Navigation Techniques for User-Friendly URLs

To create user-friendly and meaningful URLs, you can employ content navigation techniques in your Spring MVC application. These techniques include:

- **Clean URLs:** Clean URLs contain logical and descriptive path segments that represent the content being accessed. For example, `/products/electronics/mobiles` is more meaningful than `/products?id=123`.
- **SEO-friendly URLs:** Search Engine Optimization (SEO) considers URLs as a ranking factor. Creating SEO-friendly URLs with relevant keywords and proper structure can improve the visibility and ranking of your application in search engine results.
- **Slug URLs:** Slug URLs are human-readable URLs that contain a simplified version of the content title or name. For example, `/blog/my-article-title` is more user-friendly than `/blog/123`.
- **Hierarchical URLs:** Hierarchical URLs organize content in a hierarchical structure, reflecting the relationships between different content elements. For example, `/categories/electronics/mobiles` represents a hierarchical relationship between categories and subcategories.

When designing user-friendly URLs, consider using path parameters, slugs, and hierarchical structures to create meaningful URLs that are easy to understand and navigate. Ensure that the URL patterns align with the structure of your application and provide a consistent and intuitive user experience.

By leveraging the URL mapping and path matching capabilities of Spring MVC, and implementing content navigation techniques, you can create user-friendly and easily navigable URLs in your web application.

7. ConfigurableWebBindingInitializer

`ConfigurableWebBindingInitializer` is a component in Spring MVC that provides a way to customize data binding and type conversion behavior in a Spring MVC application. It allows you to configure various aspects of data binding, such as conversion rules, formatting, and validation, to handle complex data binding

scenarios.

Data Binding and Type Conversion in Spring MVC

Data binding is the process of converting request data (such as form parameters, query parameters, or request body) into Java objects, and vice versa. It allows you to seamlessly map user input to corresponding Java objects, making it easier to process and validate user input.

Type conversion, a subset of data binding, focuses on converting one data type to another. For example, converting a String value to an Integer or a Date object. Spring MVC provides a rich set of built-in type converters that can handle common type conversions automatically.

To customize data binding and type conversion in a Spring MVC application, you can use the `ConfigurableWebBindingInitializer`. It allows you to configure various components involved in the data binding process, such as `PropertyEditors`, `Converter`, and `Formatter` instances.

Handling Complex Data Binding Scenarios

In complex data binding scenarios, you may need to handle multiple input sources, nested objects, collection types, or custom conversion logic. To address such scenarios, you can customize the `ConfigurableWebBindingInitializer` as follows:

- **Handling Multiple Input Sources:** Spring MVC allows you to bind data from different sources, such as request parameters, path variables, or request bodies. You can customize the `ConfigurableWebBindingInitializer` to handle multiple input sources by registering appropriate `PropertyEditors`, `Converters`, or `Formatters` for each source.
- **Handling Nested Objects:** When binding data to nested objects, you can configure the `ConfigurableWebBindingInitializer` to recursively bind the nested properties. This can be achieved by registering nested `PropertyEditors`, `Converters`, or `Formatters` for each level of the nested object.
- **Handling Collection Types:** Spring MVC provides support for binding

collection types, such as lists, sets, or arrays. You can customize the `ConfigurableWebBindingInitializer` to handle collection types by registering appropriate `PropertyEditors`, `Converters`, or `Formatters` for the collection elements.

- **Custom Conversion Logic:** In certain cases, you may need to implement custom conversion logic beyond what can be achieved through `PropertyEditors`, `Converters`, or `Formatters`. In such scenarios, you can create your own custom data binding components and register them with the `ConfigurableWebBindingInitializer`.

8. Spring HATEOAS

Spring HATEOAS is a module of the Spring Framework that provides support for building hypermedia-driven RESTful APIs. It enables developers to follow the principles of HATEOAS (Hypermedia as the Engine of Application State) and design APIs that are self-descriptive, discoverable, and easily navigable.

HATEOAS is a principle in RESTful architecture that emphasizes the inclusion of hypermedia links in API responses. In HATEOAS, the API responses not only contain the requested data but also provide links to related resources and actions that can be performed on those resources. This makes the API self-descriptive, allowing clients to navigate and interact with the API dynamically, without relying on prior knowledge of the API's structure.

HATEOAS offers several benefits:

- **Discoverability:** With hypermedia links, clients can discover and navigate to related resources and actions dynamically, without having to hardcode or rely on out-of-band information about the API.
- **Loose Coupling:** HATEOAS promotes loose coupling between clients and servers. Clients can follow links and interact with resources based on the provided hypermedia, reducing the dependency on specific URLs or API versions.

- **Evolvability:** HATEOAS enables API evolution without breaking clients. By providing links and controlling the resource transitions, servers can introduce changes or add new capabilities while maintaining backward compatibility.

Building Hypermedia-Driven RESTful APIs using Spring HATEOAS

Spring HATEOAS provides a set of tools and abstractions for building hypermedia-driven RESTful APIs in a Spring MVC application. Here's an overview of the key features and concepts:

1. **Resource Representation Model:** Spring HATEOAS introduces the `Resource` and `Resources` classes, which encapsulate the data to be returned in an API response along with associated hypermedia links. The `Resource` class represents a single resource, while the `Resources` class represents a collection of resources.
2. **Linking Resources:** Spring HATEOAS provides the `Link` class to create hypermedia links between resources. Links can be added to resources using the `add()` method, providing a relationship name and a target URL.
3. **Controller Support:** Spring HATEOAS provides annotations like `@RestController`, `@RequestMapping`, and `@GetMapping` for building RESTful controllers. These annotations can be combined with Spring MVC annotations to define the API endpoints and return `Resource` or `Resources` objects that include hypermedia links.
4. **Link Building:** Spring HATEOAS provides a `LinkBuilder` API for creating links in a fluent and type-safe manner. `LinkBuilders` help in generating hypermedia links dynamically, based on the resource's state or the context of the request.
5. **Representation Model Assemblers:** `Representation Model Assemblers` in Spring HATEOAS help convert domain objects to `Resource` or `Resources` objects by adding hypermedia links. They provide a convenient way to map domain objects to hypermedia representations.

Linking and Navigating Resources

With Spring HATEOAS, you can link and navigate between resources using hypermedia links. Clients can follow these links to perform actions or access related resources. Some common practices include:

1. **Linking to Related Resources:** When returning a resource representation, include hypermedia links that point to related resources. For example, a response for a book resource might include links to the author, publisher, or other books by the same author.
2. **Pagination and Collection Links:** When returning a collection of resources, include links for pagination, allowing clients to navigate through different pages of the collection. Additionally, include self-links for the current page and links to the first, last, and next pages.
3. **Action-Based Links:** Include links that represent actions or operations that can be performed on a resource. For example, a link to update a resource, delete a resource, or initiate a specific action.
4. **Conditional Links:** Dynamically include or exclude links based on the state or context of the resource. For example, show a "checkout" link only if the resource is a shoppable item or show an "edit" link only if the client has the necessary permissions.

9. CORS Support

Cross-Origin Resource Sharing (CORS) is a mechanism that allows web browsers to securely make cross-origin requests from web applications. It is a browser-based security feature that ensures requests from one origin (domain, protocol, and port combination) can access resources from another origin.

By default, web browsers enforce the same-origin policy, which restricts JavaScript code running in a web page from making requests to a different origin. However, there are legitimate cases where web applications need to make cross-origin requests, such as accessing APIs from different domains.

CORS enables cross-origin requests by introducing additional HTTP headers that allow servers to specify which origins are allowed to access their resources. The browser uses these headers to determine whether a cross-origin request is safe and should be allowed.

To configure CORS support in a Spring MVC application, you can use the `@CrossOrigin` annotation or configure CORS globally. Here's an overview of the configuration options:

- **@CrossOrigin Annotation:** You can apply the `@CrossOrigin` annotation to individual controller methods or controller classes to enable CORS support for specific endpoints. The annotation allows you to specify the allowed origins, HTTP methods, request headers, and other CORS-related settings.
- **Global Configuration:** Spring MVC allows you to configure CORS globally by implementing the `WebMvcConfigurer` interface and overriding the `addCorsMappings()` method. In this method, you can define global CORS settings that apply to all endpoints in the application.

```
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://example.com")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("Content-Type")
            .exposedHeaders("Custom-Header")
            .allowCredentials(true)
            .maxAge(3600);
    }
}
```

The above example enables CORS for endpoints under the `"/api"` path, allows requests from "<http://example.com>", specifies allowed HTTP methods and headers, enables credentials, and sets the maximum age of preflight requests to 3600 seconds.

Handling Cross-Origin Requests and Security Considerations

When handling cross-origin requests, it's important to consider security implications and potential vulnerabilities. Here are some security considerations:

1. **Whitelist Allowed Origins:** Only allow trusted origins to access your resources by specifying specific domains in the CORS configuration. Avoid using wildcard `*` unless it's necessary.
2. **Restrict Allowed Methods and Headers:** Specify the allowed HTTP methods and headers explicitly to prevent unauthorized access or abuse.
3. **Enable CORS only for Required Endpoints:** Enable CORS only for the specific endpoints that require cross-origin access. Limiting CORS to necessary endpoints reduces the attack surface.
4. **Proper Authentication and Authorization:** Ensure that authentication and authorization mechanisms are in place to prevent unauthorized access to sensitive resources.
5. **CSRF Protection:** Cross-Site Request Forgery (CSRF) attacks can still occur even with CORS enabled. Implement CSRF protection measures, such as using CSRF tokens, to mitigate this risk.
6. **Content Security Policy (CSP):** Implement a Content Security Policy to restrict the types of content that can be loaded by your web application, reducing the impact of cross-origin requests.